

Lexer, Token & Keyword Design

COOL Compiler — frontend/lexer

Why the Scanner was designed the way it was

A hand-written lexer needs three recurring capabilities at every single point in the source: look at the current character without consuming it, look one character further ahead without consuming anything, and consume-and-advance while tracking line number. Rather than have each token-scanning branch reimplement this bookkeeping inline (which is exactly what caused early, discarded drafts of this lexer to forget to advance the cursor, or forget to increment the line counter on newlines), a single small Scanner struct plus four tiny helper functions were introduced once and reused everywhere:

```
typedef struct Scanner {
    const char *buffer;
    size_t size;
    size_t pos;
    int line;
} Scanner;

static char peek(Scanner *s);      /* current char, no consume */
static char peek_next(Scanner *s); /* lookahead char, no consume */
static char advance(Scanner *s);   /* consume current, track \n */
static bool at_end(Scanner *s);    /* end-of-buffer check */
```

- `advance()` is the **only** place `pos` is mutated and the **only** place `line` is incremented. No category-scanning function is allowed to touch either field directly, which is exactly what guarantees line tracking can never silently drift out of sync with position.
- Genuine lookahead (`peek_next`) is reserved for the handful of places that actually need it — deciding between `<`, `<=`, and `<=;`; deciding whether `(` starts a block comment; deciding an escape sequence inside a string. It is never misapplied to tokens that do not need it (e.g. `+` and `-` are valid regardless of what follows them).
- This mirrors the category-based dispatch designed early on (a single `q0` branching into single-char operators, two-char operators, comments, strings, numbers, and identifiers/keywords): each category becomes its own small function built on the shared cursor, rather than one long, undifferentiated chain of special cases.

Ownership: how each piece of data is created, used, and released

Component	Created by	Used by	Destroyed by
Source buffer	<code>main.c</code> (reads the whole file once)	Scanner, read-only, via <code>peek/advance</code>	<code>main.c</code> , after <code>lex()</code> returns
Scanner	<code>lex()</code> , on the stack, per call	every <code>scan_*</code> function, by pointer	Falls out of scope automatically; nothing heap-allocated

Component	Created by	Used by	Destroyed by
StrBuf (scratch text buffer)	Each scan_string / scan_number / scan_identifier, on the stack	Accumulates characters for one token only	Freed by the same function that created it, before every single return path — ownership never transfers out
Entry* (interned identifier/string/int)	InternTables, via table_add_entry	Copied by value into a Token as its value field	InternTables (owns every Entry it created), destroyed once at the very end
Token.lexeme (error text only)	strdup() inside the error-producing function, for TOKEN_ERROR only	report_lex_error(), for printing	tokenlist_destroy(), which frees every non-NULL lexeme
TokenList	lex(), via tokenlist_create()	Every subsequent phase (parser, eventually)	main.c, after every phase that needs it has finished

The rule that keeps this consistent: a scratch buffer's owner never changes hands mid-function depending on which branch executes. If a piece of data needs to outlive the function that built it (as an error token's text does), an explicit, independent copy is made (`strdup`) at the moment of handoff, and the original scratch buffer is *still* freed by its own function on every path, success or error alike. This was specifically corrected during development after an earlier draft let ownership of the scratch buffer implicitly transfer to the token on error but not on success — an asymmetry that is fragile because nothing in the type system enforces it.

Token: what it stores, and what it deliberately does not

```
typedef struct Token {
    TokenType type;
    Entry *value;          /* NULL for keywords/punctuation */
    int line;
    char *lexeme;          /* NULL except for TOKEN_ERROR */
    LexErrorKind error_kind; /* only meaningful when type == TOKEN_ERROR */
} Token;
```

- `value` is populated only for the token types that were actually interned (`OBJECT_ID`, `TYPE_ID`, `INT_CONST`, `STR_CONST`). This is the entire payoff of interning: two tokens for the same identifier carry the identical pointer, so later phases compare them in $O(1)$ rather than re-comparing text.
- `lexeme` is deliberately left `NULL` for every token type that already has an `Entry*` — `entry_str(value)` already gives back the exact same text on demand, so storing it a second time would be pure duplication. It is populated only for `TOKEN_ERROR`, which has no `Entry*` at all (invalid text is never interned), making `lexeme` the only surviving record of what was actually seen.
- `error_kind` exists as its own small enum rather than encoding the error category as a formatted string inside `lexeme`. A fixed, small vocabulary of error kinds is exactly what an enum is for — it is type-checked at compile time, requires no parsing on the reporting side, and keeps `lexeme` meaning exactly one thing (the raw offending text) regardless of token type.

TokenList: growable storage, same discipline as InternTables

A plain array cannot be used directly because the number of tokens in a source file is not known in advance. `TokenList` is a small growable-array abstraction (create / append / destroy) that doubles its backing array's capacity whenever it fills, following the same "caller never sees the growth mechanics" discipline used by the tables: the lexer only ever calls `tokenlist_append(list, tok)` and never needs to know or care about capacity, realloc, or doubling happening underneath.

Ownership handoff: the lexer creates and populates the `TokenList`, but the driver (`main.c`) is the one that ultimately owns and destroys it — matching the pattern used for `InternTables`. This lets the same `TokenList` be handed onward to the parser before the driver eventually frees it, once every phase that needs it has finished.

Keyword lookup: a compiled-in table, not a runtime file

COOL's reserved words are fixed and finite — part of the language definition, never something the user's program can change. Rather than reading a keyword list from disk at runtime (an unnecessary dependency and I/O cost for static data), the keyword set is a small, compiled-in static array of `{text, TokenType}` pairs, checked with a case-insensitive comparison, consistent with COOL keywords being case-insensitive by specification. `true` and `false` are handled as a special case outside this table, since COOL specifically requires their first letter to be lowercase — a rule this general case-insensitive table cannot express by itself.

The lookup is only ever performed **after** an entire identifier has already been fully scanned (maximal munch), never mid-scan. This specifically avoids a subtle but serious correctness bug: if the scanner tried to match keyword prefixes as it went, an identifier like `iffy` could be wrongly split into the keyword `IF` followed by a stray identifier `fy`. Scanning the whole identifier first, and only then checking it against the keyword table, guarantees this can never happen.

How the lexer handles errors efficiently

- **Detection happens exactly once, at the only point it is knowable** — only the character-scanning phase has access to the raw text needed to notice an illegal character, an unterminated string, an unterminated (possibly nested) comment, an illegal escape sequence, or an over-long string constant.
- **The lexer never halts the whole compilation on the first error.** Rather than calling `exit()` the moment something invalid is seen, the lexer encodes the problem as an ordinary `TOKEN_ERROR` token (with its `error_kind` and offending lexeme attached) and appends it to the token list exactly like any other token, then keeps scanning. This mirrors how real compilers behave: a program with two unrelated typos gets both reported in one pass, rather than the user fixing one, recompiling, and only then discovering the second.
- **Reporting is a completely separate responsibility from detection.** The lexer never prints anything itself. A single pass over the finished `TokenList`, run by the driver (`report_lex_error`), finds every `TOKEN_ERROR` and prints a message for each, using `error_kind` to index directly into a static lookup table of human-readable strings — an $O(1)$ array index, not string parsing.
- **Nested block comments are tracked with a counter, not re-scanned recursively.** A single depth integer, incremented on every `( *` and decremented on every `*)`, correctly handles arbitrary nesting depth in one linear pass, with an unterminated comment detected the instant end-of-buffer is reached while depth is still greater than zero — and the line number reported is the comment's *opening* line, not wherever end-of-buffer happens to fall, since that is what actually helps the user find the mistake.

- **The driver, not the lexer, decides overall policy.** Whether to proceed to parsing after lexical errors are found, and what exit status to return, is a decision made once by `main.c` after seeing the complete picture — never hardcoded inside the lexer itself.

What the lexer deliberately never checks

Consistent with keeping the lexer strictly a character-to-token translator, several classes of "wrongness" are deliberately left to later phases:

- Whether two adjacent, individually-valid tokens form a legal sequence (e.g. `<+` lexes as two perfectly ordinary tokens, `TOKEN_LT` then `TOKEN_PLUS` — whether that sequence is grammatically legal is the parser's job).
- Whether brackets/braces/parentheses are balanced across the whole program (a parser-level, grammar-driven concern, typically enforced implicitly by the shape of the grammar itself).
- Whether an identifier is declared, in scope, or has a consistent type (semantic analysis, using a completely different, scoped lookup structure built later).
- Whether an integer literal's numeric value overflows the target's representable range (the lexer only interns the literal's raw text; range validation happens in semantic analysis, which has the context to produce a proper diagnostic).